

*Istituto **T**ecnico **E**conomico e **T**ecnologico “**Girolamo Caruso**”
di Alcamo*

Classe: 5^a sezione BIT

Indirizzo: **Informatica e Telecomunicazioni**
Articolazione: **Informatica**

Anno Scolastico **2025 – 2026**

Compito di realtà: “Sito 15 Maggio”
Materia: **Informatica**

DOCUMENTO DI PROGETTO
DEL SITO WEB DELL’AZIENDA

“Giudmunna”

REALIZZATO

DA

Munna Giuseppe Davide

FIRMA

Giuseppe Davide Munna

Classe: 5ª BIT

Nome del progetto: Giudmunna – E-commerce iPhone Ricondizionati

Versione: 01

Data del progetto: 15/05

Autore del progetto: Munna Giuseppe Davide

REQUISITI DI PROGETTO (PROBLEMA)

Realizzare un sito web dinamico per la vendita online di smartphone iPhone ricondizionati. Il sito deve permettere la consultazione del catalogo prodotti, la registrazione degli utenti, l'autenticazione tramite login, la gestione dei dati anagrafici, la selezione dei prodotti tramite carrello e la registrazione degli ordini effettuati. Il sistema deve inoltre prevedere una semplice area amministrativa per l'inserimento di nuovi prodotti e la visualizzazione degli ordini.

ANALISI DEI REQUISITI

In base ai requisiti di progetto, si prevede la realizzazione di un sito web dinamico con struttura gerarchica. Il sito distingue tre tipologie di utenti:

- Utente non registrato: può visualizzare il catalogo prodotti.
- Utente registrato: può effettuare il login, gestire il proprio profilo, aggiungere prodotti al carrello ed effettuare ordini.
- Amministratore: può inserire nuovi prodotti nel catalogo e visualizzare gli ordini effettuati.

Le funzionalità principali del sistema sono:

- registrazione utenti
- autenticazione (login/logout)
- visualizzazione catalogo prodotti
- gestione profilo utente
- gestione carrello
- registrazione ordini
- area amministrativa

MAPPA DEL SITO

grazie all header ogni pagina si collega direttamente a tutte le altre

- index.php
Home page con prodotti in evidenza
- catalogo.php
Elenco completo prodotti
- prodotto.php
Dettaglio singolo prodotto e varianti
- registrazione.php
Creazione account (username, email, password)
- login.php
Accesso utente
- logout.php
Uscita dalla sessione
- profilo.php
Gestione dati anagrafici cliente
- carrello.php
Gestione articoli selezionati
- checkout.php
Conferma ordine da carrello e salvataggio nel database
- ordine.php
Ordine rapido di un singolo prodotto
- miei_ordini.php
Storico ordini dell'utente
- chi-siamo.php
Presentazione dell'attività e posizione
- sostenibilita.php
Pagina informativa sulla sostenibilità
- admin_prodotti.php
Area amministratore per inserimento e gestione prodotti
- admin_ordini.php
Area amministratore per visualizzazione ordini
- header.php
Intestazione comune con logo e menu di navigazione

- footer.php
Piè di pagina comune con dicitura obbligatoria
- config.php
Configurazione database, gestione sessioni e funzioni condivise
- style.css
Foglio di stile per la grafica del sito
- js/site.js
Script JavaScript per interazioni, validazioni e menu dinamico

STRUTTURA GRAFICA DEL SITO

Tutte le pagine del sito presentano una struttura comune composta da:

- Header: contenente logo e menu di navigazione
- Area centrale: contenuto specifico della pagina
- Footer: contenente la dicitura obbligatoria

"Sito web realizzato per motivi didattici e non a fini commerciali"

TECNOLOGIE UTILIZZATE

Il sito è stato sviluppato utilizzando le seguenti tecnologie:

- HTML per la struttura delle pagine web
- CSS per la presentazione grafica
- JavaScript per l'interazione lato client
- PHP per la gestione della logica lato server
- MySQL per la gestione del database

FUNZIONALITÀ IMPLEMENTATE

- la registrazione di nuovi utenti
- l'autenticazione tramite login
- la gestione dei dati anagrafici
- la visualizzazione del catalogo prodotti
- la consultazione del dettaglio prodotto
- l'aggiunta di prodotti al carrello
- la registrazione degli ordini
- la gestione amministrativa dei prodotti e degli ordini

Progettazione del database per il progetto Giudmunna

PROBLEMA (realta di riferimento)

Il committente vuole realizzare un database a supporto del progetto Giudmunna, una web application di e-commerce dedicata alla vendita di iPhone ricondizionati. Il sistema deve permettere la gestione degli utenti registrati, dei dati anagrafici e di spedizione dei clienti, del catalogo dei prodotti organizzato per varianti e del processo di acquisto composto da carrello, conferma ordine e consultazione dello storico ordini. Deve inoltre essere prevista una semplice area amministrativa per l'inserimento di nuove varianti di prodotto e per la visualizzazione degli ordini ricevuti.

ANALISI

Analizzando la realtà di riferimento si osserva che il dominio non riguarda il singolo smartphone generico, ma una variante di prodotto definita dalla combinazione fra modello, capacità di memoria, colore e grado estetico. Il database deve quindi evitare ridondanze e permettere di rappresentare correttamente tali combinazioni, oltre a supportare la registrazione degli utenti e la gestione degli ordini.

a) Rielaborazione della realtà di riferimento con ipotesi progettuali:

- Ogni utente può registrarsi al sito con username, email, password e ruolo applicativo;
- Un utente che effettua acquisti deve poter completare il proprio profilo cliente con nome, cognome, indirizzo, città, CAP e telefono;
- Ogni cliente corrisponde ad un solo utente registrato, mentre un utente può avere al più un profilo cliente;
- Il catalogo è composto da prodotti ricondizionati e ogni prodotto rappresenta una specifica variante commerciale;
- Una variante è identificata univocamente dalla combinazione modello-capacità-colore-grado estetico;
- Un cliente può effettuare nel tempo più ordini e ciascun ordine può contenere una o più righe d'ordine;
- Ogni riga d'ordine fa riferimento ad un solo prodotto e ne memorizza quantità e prezzo unitario applicato al momento dell'acquisto;
- Un amministratore può inserire nuove varianti nel catalogo e consultare tutti gli ordini presenti nel sistema.

b) Tipo di applicazione e hardware ipotizzato:

Si ipotizza una web application eseguita su stack locale o su server web LAMP/WAMP/XAMPP. L'applicazione può essere utilizzata da browser desktop o mobile collegati in rete locale o via Internet.

Per lo sviluppo è sufficiente un personal computer con web server, interprete PHP e DBMS MySQL/MariaDB.

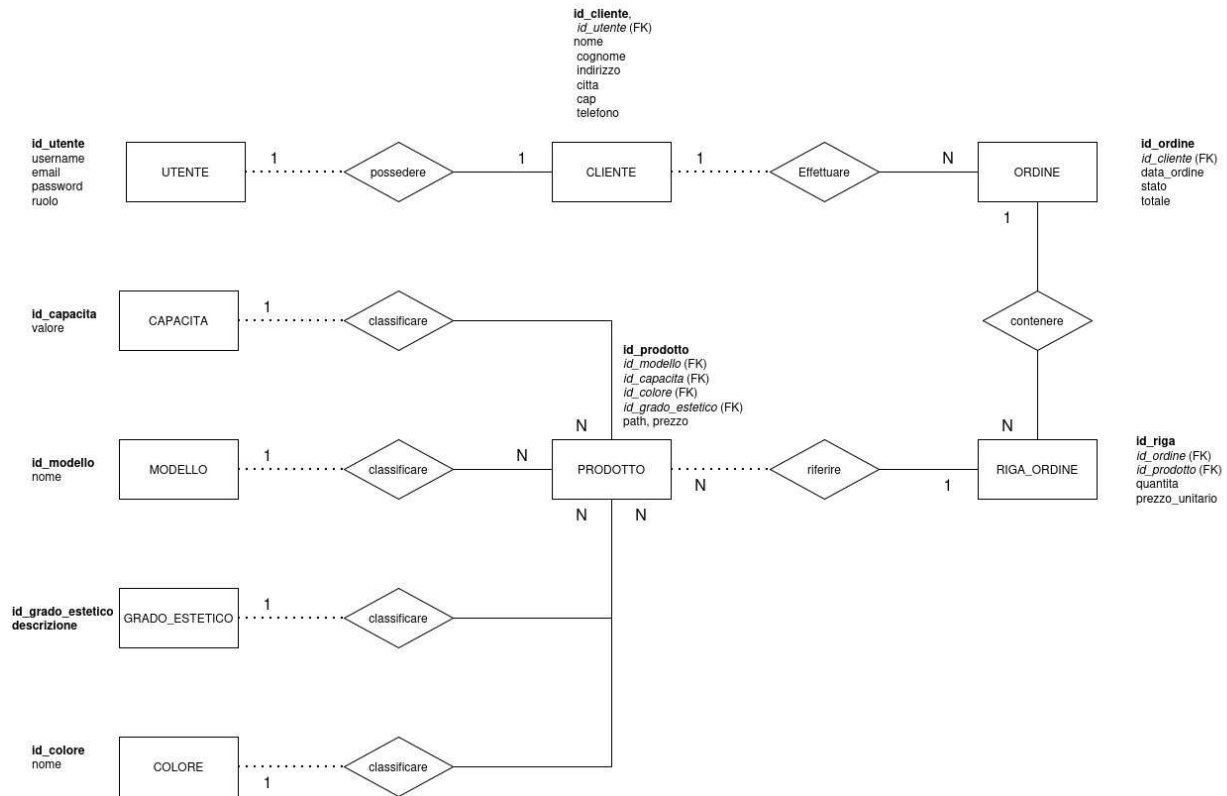
c) Tipo di software utilizzato:

- DBMS relazionale MySQL o MariaDB;
- server web Apache;
- interprete lato server PHP;
- browser web per l'interazione con il sistema;
- eventuali strumenti di amministrazione DB come phpMyAdmin per importazione ed esecuzione dello script SQL.

d) Linguaggi utilizzati:

- SQL per definizione dello schema, vincoli, inserimento e interrogazione dei dati;
- PHP per la logica applicativa lato server e l'interazione con il database;
- HTML e CSS per la struttura e la presentazione delle pagine web;
- JavaScript solo come supporto lato client, non essenziale al modello dati.

MODELLO CONCETTUALE (Realizzazione dello schema E/R e del suo documento di lettura)



1) Schema E/R

Lo schema E/R individuato per il progetto comprende le seguenti entità principali: UTENTE, CLIENTE, MODELLO, CAPACITA, COLORE, GRADO_ESTETICO, PRODOTTO, ORDINE e RIGA_ORDINE.

Le associazioni fondamentali sono: UTENTE possiede CLIENTE, CLIENTE effettua ORDINE, ORDINE contiene RIGA_ORDINE, RIGA_ORDINE fa riferimento a PRODOTTO, mentre PRODOTTO è classificato da MODELLO, CAPACITA, COLORE e GRADO_ESTETICO.

2) Documento di lettura dello schema E/R

Entità individuate:

- UTENTE: rappresenta l'account di accesso al sito e distingue utenti normali e amministratori.
- CLIENTE: contiene i dati anagrafici e di spedizione associati ad un utente registrato.
- MODELLO: identifica il modello commerciale dello smartphone, ad esempio iPhone 13 o iPhone 15 Pro.
- CAPACITA: contiene i tagli di memoria disponibili, ad esempio 128 GB o 256 GB.
- COLORE: contiene i colori associabili alle varianti di prodotto.

- GRADO_ESTETICO: rappresenta lo stato estetico del ricondizionato, ad esempio Buono, Ottimo o Eccellente.
- PRODOTTO: rappresenta una variante vendibile del catalogo con immagine e prezzo.
- ORDINE: rappresenta un acquisto effettuato da un cliente in una certa data con stato e totale.
- RIGA_ORDINE: rappresenta il dettaglio di ciascun articolo contenuto in un ordine.

Attributi principali delle entità:

- UTENTE: id_utente, username, email, password, ruolo.
- CLIENTE: id_cliente, id_utente, nome, cognome, indirizzo, citta, cap, telefono.
- MODELLO: id_modello, nome.
- CAPACITA: id_capacita, valore.
- COLORE: id_colore, nome.
- GRADO_ESTETICO: id_grado_estetico, descrizione.
- PRODOTTO: id_prodotto, id_modello, id_capacita, id_colore, id_grado_estetico, path, prezzo.
- ORDINE: id_ordine, id_cliente, data_ordine, stato, totale.
- RIGA_ORDINE: id_riga, id_ordine, id_prodotto, quantita, prezzo_unitario.

Associazioni, cardinalita e partecipazione:

Possedere- UTENTE-CLIENTE: 1:1. Un utente puo avere al più un profilo cliente; ogni cliente deve riferirsi a un solo utente. Partecipazione totale di CLIENTE e parziale di UTENTE.

Effettuare - CLIENTE-ORDINE: 1:N. Un cliente puo effettuare molti ordini; ogni ordine appartiene a un solo cliente. Partecipazione parziale di CLIENTE e totale di ORDINE.

Contenere - ORDINE-RIGA_ORDINE: 1:N. Un ordine contiene una o più righe; ogni riga appartiene a un solo ordine. Partecipazione totale di RIGA_ORDINE e totale di ORDINE nel caso di ordine confermato.

- PRODOTTO-RIGA_ORDINE: 1:N. Un prodotto puo comparire in molte righe d'ordine; ogni riga d'ordine si riferisce a un solo prodotto. Partecipazione totale di RIGA_ORDINE e parziale di PRODOTTO.

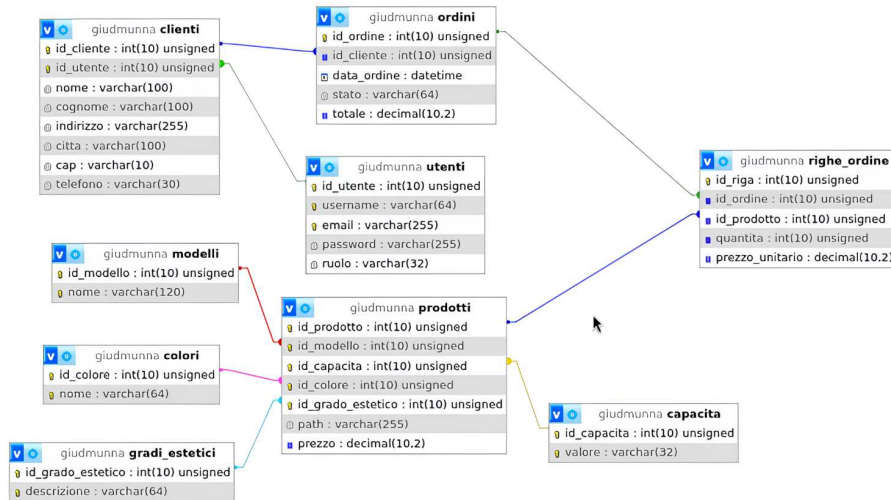
- MODELLO-PRODOTO, CAPACITA-PRODOTO, COLORE-PRODOTO, GRADO_ESTETICO-PRODOTO: ciascuna e un'associazione 1:N, perché una stessa voce di classificazione puo essere usata in molte varianti di prodotto, mentre ogni prodotto usa un solo valore per ciascuna classificazione.

Vincoli semantici rilevanti:

- username ed email devono essere univoci nel sistema;
- la combinazione id_modello, id_capacita, id_colore e id_grado_estetico deve essere univoca per evitare duplicati di variante;
- quantita deve essere positiva;
- totale e prezzo_unitario devono essere valori monetari non negativi;
- non e possibile registrare un ordine senza un cliente associato.

Associazione	Cardinalità	Partecipazione (E1 / E2)	Note e Motivo
Possedere	1:1	Parziale / Totale	Un utente può non avere ancora un profilo cliente (parziale), ma il cliente deve essere legato a un utente (totale).
Effettuare	1:N	Parziale / Totale	Un cliente può essere registrato senza aver fatto ordini (parziale), mentre l'ordine richiede un cliente (totale).
Contenere	1:N	Totale / Totale	Un ordine senza righe non ha senso di esistere (totale), e ogni riga deve far parte di un ordine (totale).
Riferire	1:N	Parziale / Totale	Un prodotto può non essere ancora stato inserito in un ordine (parziale), ma ogni riga ordine deve indicare un prodotto (totale).
Classificare	1:N	Parziale / Totale	Una categoria (modello, colore, ecc.) può esistere a catalogo anche se non usata (parziale), ma il prodotto deve avere i suoi valori obbligatori (totale).

MODELLO LOGICO-RELAZIONALE (Realizzazione dello schema logico e del suo documento di lettura)



Schema logico-relazionale:

- UTENTI(id_utente PK, username UQ, email UQ, password, ruolo)
- CLIENTI(id_cliente PK, id_utente FK UQ -> UTENTI.id_utente, nome, cognome, indirizzo, citta, cap, telefono)
- MODELLI(id_modello PK, nome UQ)
- CAPACITA(id_capacita PK, valore UQ)
- COLORI(id_colore PK, nome UQ)
- GRADI_ESTETICI(id_grado_estetico PK, descrizione UQ)
- PRODOTTI(id_prodotto PK, id_modello FK -> MODELLI.id_modello, id_capacita FK -> CAPACITA.id_capacita, id_colore FK -> COLORI.id_colore, id_grado_estetico FK -> GRADI_ESTETICI.id_grado_estetico, path, prezzo, UQ(id_modello, id_capacita, id_colore, id_grado_estetico))
- ORDINI(id_ordine PK, id_cliente FK -> CLIENTI.id_cliente, data_ordine, stato, totale)
- RIGHE_ORDINE(id_riga PK, id_ordine FK -> ORDINI.id_ordine, id_prodotto FK -> PRODOTTI.id_prodotto, quantita, prezzo_unitario)

Commenti sulla derivazione dal modello E/R:

- l'associazione 1:1 tra UTENTE e CLIENTE è stata realizzata inserendo in CLIENTI la chiave esterna id_utente con vincolo UNIQUE;
- le associazioni 1:N sono state tradotte inserendo la chiave primaria del lato uno come chiave esterna nel lato molti;
- l'entità PRODOTTO è stata normalizzata separando le informazioni ripetitive in tabelle di dominio indipendenti (MODELLI, CAPACITA, COLORI, GRADI_ESTETICI);
- l'associazione ORDINE-PRODOTTO di tipo N:M è stata risolta tramite la tabella RIGHE_ORDINE, che contiene anche gli attributi quantita e prezzo_unitario;
- la presenza del prezzo_unitario in RIGHE_ORDINE consente di conservare il valore storico applicato al momento dell'acquisto anche se il prezzo del prodotto cambia successivamente.

Struttura delle tabelle del modello

Campo	Tipo	Vincoli	Descrizione
UTENTI			
id_utente	INT UNSIGNED	PK, AUTO_INCREMENT	Identificativo univoco utente
username	VARCHAR(64)	UNIQUE, NOT NULL	Nome utente
email	VARCHAR(255)	UNIQUE, NOT NULL	Email
password	VARCHAR(255)	NOT NULL	Password hash
ruolo	VARCHAR(32)	NOT NULL	Ruolo (utente/admin)
CLIENTI			
id_cliente	INT UNSIGNED	PK, AUTO_INCREMENT	Identificativo cliente
id_utente	INT UNSIGNED	FK, UNIQUE, NOT NULL	Collegamento a UTENTI (1:1)
nome	VARCHAR(100)	NOT NULL	Nome
cognome	VARCHAR(100)	NOT NULL	Cognome
indirizzo	VARCHAR(255)	NOT NULL	Indirizzo
citta	VARCHAR(100)	NOT NULL	Città
cap	VARCHAR(10)	NOT NULL	CAP
telefono	VARCHAR(30)	NOT NULL	Telefono
MODELLI			
id_modello	INT	PK	Identificativo modello
nome	VARCHAR(120)	UNIQUE	Modello commerciale
CAPACITA			
id_capacita	INT	PK	Identificativo capacità
valore	VARCHAR(32)	UNIQUE	Taglio memoria
COLORI			
id_colore	INT	PK	Identificativo colore
nome	VARCHAR(64)	UNIQUE	Colore

GRADI_ESTETICI			
id_grado_estetico	INT	PK	Identificativo grado
descrizione	VARCHAR(64)	UNIQUE	Condizione estetica
PRODOTTI			
id_prodotto	INT UNSIGNED	PK, AUTO_INCREMENT	Identificativo prodotto
id_modello	INT UNSIGNED	FK, NOT NULL	FK → MODELLI
id_capacita	INT UNSIGNED	FK, NOT NULL	FK → CAPACITA
id_colore	INT UNSIGNED	FK, NOT NULL	FK → COLORI
id_grado_estetico	INT UNSIGNED	FK, NOT NULL	FK → GRADI_ESTETICI
path	VARCHAR(255)	NOT NULL	URL immagine
prezzo	DECIMAL(10,2)	NOT NULL	Prezzo
ORDINI			
id_ordine	INT UNSIGNED	PK, AUTO_INCREMENT	Identificativo ordine
id_cliente	INT UNSIGNED	FK, NOT NULL	Cliente
data_ordine	DATETIME	NOT NULL	Data ordine
stato	VARCHAR(64)	NOT NULL	Stato ordine
totale	DECIMAL(10,2)	NOT NULL	Totale
RIGHE_ORDINE			
id_riga	INT UNSIGNED	PK, AUTO_INCREMENT	Identificativo riga
id_ordine	INT UNSIGNED	FK, NOT NULL	FK → ORDINI
id_prodotto	INT UNSIGNED	FK, NOT NULL	FK → PRODOTTI
quantita	INT UNSIGNED	NOT NULL	Quantità
prezzo_unitario	DECIMAL(10,2)	NOT NULL	Prezzo per pezzo

Chiarimenti sui vincoli di referenzialità:

- la cancellazione di un utente comporta la cancellazione del relativo cliente tramite ON DELETE CASCADE;
- la cancellazione di un ordine elimina automaticamente le sue righe tramite ON DELETE CASCADE;
- la cancellazione di un prodotto referenziato in una riga d'ordine non è consentita, così da preservare la coerenza storica degli acquisti;
- la cancellazione di un cliente con ordini associati è bloccata tramite ON DELETE RESTRICT.

MODELLO FISICO (Definizione della struttura delle tabelle, istruzioni per la creazione, manipolazione e interrogazione del database)

istruzioni di creazione:

```
CREATE TABLE utenti (  
  id_utente INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  username VARCHAR(64) NOT NULL,  
  email VARCHAR(255) NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  ruolo VARCHAR(32) NOT NULL DEFAULT 'utente',  
  PRIMARY KEY (id_utente),  
  UNIQUE KEY uk_utenti_username (username),  
  UNIQUE KEY uk_utenti_email (email)  
);
```

```
CREATE TABLE prodotti (  
  id_prodotto INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  id_modello INT UNSIGNED NOT NULL,  
  id_capacita INT UNSIGNED NOT NULL,  
  id_colore INT UNSIGNED NOT NULL,  
  id_grado_estetico INT UNSIGNED NOT NULL,  
  path VARCHAR(255) NOT NULL DEFAULT '',  
  prezzo DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
  PRIMARY KEY (id_prodotto),  
  UNIQUE KEY uk_prodotto_variante (id_modello, id_capacita, id_colore, id_grado_estetico)  
);
```

```
CREATE TABLE ordini (  
  id_ordine INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  id_cliente INT UNSIGNED NOT NULL,  
  data_ordine DATETIME NOT NULL,  
  stato VARCHAR(64) NOT NULL DEFAULT 'In lavorazione',  
  totale DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
  PRIMARY KEY (id_ordine)  
);
```

```
CREATE TABLE righe_ordine (  
  id_riga INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  id_ordine INT UNSIGNED NOT NULL,  
  id_prodotto INT UNSIGNED NOT NULL,  
  quantita INT UNSIGNED NOT NULL DEFAULT 1,  
  prezzo_unitario DECIMAL(10,2) NOT NULL,  
  PRIMARY KEY (id_riga)  
);
```

Esempi di manipolazione dei dati:

- registrazione di un nuovo utente con INSERT INTO utenti;
- inserimento o aggiornamento del profilo anagrafico cliente con INSERT o UPDATE sulla tabella clienti;
- inserimento di una nuova variante prodotto dall'area admin con INSERT INTO prodotti;
- creazione ordine con INSERT INTO ordini seguito dagli INSERT in righe_ordine;
- consultazione degli ordini cliente e amministratore con query JOIN tra ordini, righe_ordine, prodotti e tabelle di classificazione.

Principali query interrogative usate

config.php

```
SHOW COLUMNS FROM prodotti LIKE 'path'
```

index.php

```
SELECT p.id_prodotto, m.nome AS modello, c.valore AS capacita, co.nome AS colore,  
      g.descrizione AS grado_estetico, p.path, p.prezzo  
FROM prodotti p  
JOIN modelli m ON p.id_modello = m.id_modello  
JOIN capacita c ON p.id_capacita = c.id_capacita  
JOIN colori co ON p.id_colore = co.id_colore  
JOIN gradi_estetici g ON p.id_grado_estetico = g.id_grado_estetico  
ORDER BY p.id_prodotto ASC LIMIT 3
```

catalogo.php

```
SELECT p.id_prodotto, m.nome AS modello, c.valore AS capacita, co.nome AS colore,  
      g.descrizione AS grado_estetico, p.path, p.prezzo  
FROM prodotti p  
JOIN modelli m ON p.id_modello = m.id_modello  
JOIN capacita c ON p.id_capacita = c.id_capacita  
JOIN colori co ON p.id_colore = co.id_colore  
JOIN gradi_estetici g ON p.id_grado_estetico = g.id_grado_estetico  
ORDER BY modello, capacita, colore, grado_estetico, p.id_prodotto
```

profilo.php

- **SELECT** id_cliente, nome, cognome, indirizzo, citta, cap, telefono
 FROM clienti WHERE id_utente = ?
- **UPDATE** clienti SET nome=?, cognome=?, indirizzo=?, citta=?, cap=?, telefono=?
 WHERE id_cliente=?
- **INSERT INTO** clienti (id_utente, nome, cognome, indirizzo, citta, cap, telefono)
 VALUES (?, ?, ?, ?, ?, ?, ?)

registrazione.php

```
INSERT INTO utenti (username, email, password) VALUES (?, ?, ?)
```

login.php

SELECT id_utente, password FROM utenti WHERE username = ?

prodotto.php

- **SELECT** p.id_prodotto, m.nome AS modello, c.valore AS capacita, co.nome AS colore, g.descrizione AS grado_estetico, p.path, p.prezzo
FROM prodotti p
JOIN modelli m ON p.id_modello = m.id_modello
JOIN capacita c ON p.id_capacita = c.id_capacita
JOIN colori co ON p.id_colore = co.id_colore
JOIN gradi_estetici g ON p.id_grado_estetico = g.id_grado_estetico
WHERE p.id_prodotto = ?
- **SELECT** p.id_prodotto, m.nome AS modello, c.valore AS capacita, co.nome AS colore, g.descrizione AS grado_estetico, p.path, p.prezzo
FROM prodotti p
JOIN modelli m ON p.id_modello = m.id_modello
JOIN capacita c ON p.id_capacita = c.id_capacita
JOIN colori co ON p.id_colore = co.id_colore
JOIN gradi_estetici g ON p.id_grado_estetico = g.id_grado_estetico
WHERE m.nome = ? ORDER BY c.valore, co.nome, g.descrizione

carrello.php

- **SELECT** p.id_prodotto, m.nome AS modello, c.valore AS capacita, co.nome AS colore, g.descrizione AS grado_estetico, p.path, p.prezzo
FROM prodotti p
JOIN modelli m ON p.id_modello = m.id_modello
JOIN capacita c ON p.id_capacita = c.id_capacita
JOIN colori co ON p.id_colore = co.id_colore
JOIN gradi_estetici g ON p.id_grado_estetico = g.id_grado_estetico
WHERE p.id_prodotto = ?

ordine.php

- **SELECT** id_cliente FROM clienti WHERE id_utente = ?
- **SELECT** m.nome AS modello, c.valore AS capacita, co.nome AS colore, g.descrizione AS grado_estetico, p.prezzo
FROM prodotti p
JOIN modelli m ON p.id_modello = m.id_modello
JOIN capacita c ON p.id_capacita = c.id_capacita
JOIN colori co ON p.id_colore = co.id_colore
JOIN gradi_estetici g ON p.id_grado_estetico = g.id_grado_estetico
WHERE p.id_prodotto = ?
- **INSERT INTO** ordini (id_cliente, data_ordine, stato, totale) VALUES (?, ?, ?, ?)
- **INSERT INTO** righe_ordine (id_ordine, id_prodotto, quantita, prezzo_unitario) VALUES (?, ?, ?, ?)

checkout.php

- **SELECT** id_cliente FROM clienti WHERE id_utente = ?
- **SELECT** p.id_prodotto, m.nome AS modello, p.prezzo
FROM prodotti p
JOIN modelli m ON p.id_modello = m.id_modello
WHERE p.id_prodotto = ?
- **INSERT INTO** ordini (id_cliente, data_ordine, stato, totale) VALUES (?, ?, ?, ?)
- **INSERT INTO** righe_ordine (id_ordine, id_prodotto, quantita, prezzo_unitario)
VALUES (?, ?, ?, ?)

Link file github : <https://github.com/giudmunna/sito15Maggio>

1. Sanitizzazione input e output

1.1 Sanitizzazione e validazione lato server

Nelle pagine `login.php` e `registrazione.php` vengono ripuliti i dati inviati dal form:

```
```php
$username = trim($_POST['username'] ?? '');
$email = trim($_POST['email'] ?? '');
$password = trim($_POST['password'] ?? '');
```
```

Questa operazione elimina spazi inutili prima e dopo il valore.

1.2 Escape dell'output HTML

Per evitare cross-site scripting (XSS) viene usata la funzione `htmlspecialchars()` quando si stampano dati dinamici nell'HTML:

```
```php
<p class="messaggio"><?php echo htmlspecialchars($messaggio); ?></p>
```
```

Esempi in molte pagine:

- `catalogo.php`
- `prodotto.php`
- `header.php`
- `carrello.php`
- `checkout.php`

Un helper dedicato è definito in `admin\_prodotti.php`:

```
```php
function h(string $s): string
{
 return htmlspecialchars($s, ENT_QUOTES, 'UTF-8');
}
```
```

Esempio JavaScript di protezione XSS:

```
```javascript
const userMessage = getMessageFromServer();
const target = document.getElementById('messaggio');
if (target) {
 // Usa.textContent per inserire testo senza interpretare HTML.
 target.textContent = userMessage;
}
```
```

Questo evita di eseguire codice HTML/JavaScript fornito dall'utente, proteggendo la pagina da XSS.

1.3 Validazione di campi specifici

Nel pannello di amministrazione esiste una funzione che normalizza il prezzo inserito:

```
```php
function normalize_price(string $input): ?float
{
 $input = trim($input);
 if ($input === '') {
 return null;
 }
 if (strlen($input) > 32) {
 return null;
 }
 $input = str_replace(['€', ' '], '', $input);
 $input = str_replace(',', '.', $input);
 if (!preg_match('/^(?:0|[1-9]\d*)(?:\.\d{1,2})?$/i', $input)) {
 return null;
 }
 $p = (float)$input;
 if ($p <= 0 || $p > 999999.99) {
 return null;
 }
 return round($p, 2);
}
```
```

Questa funzione accetta solo numeri validi e converte la virgola in punto.

1.4 Validazione URL immagini

Per i link alle immagini è presente questa validazione:

```
```php
function is_valid_image_url(string $url): bool
{
 $url = trim($url);
 if ($url === '') {
 return false;
 }
 if (!filter_var($url, FILTER_VALIDATE_URL)) {
 return false;
 }
 $scheme = parse_url($url, PHP_URL_SCHEME);
 if (!in_array(strtolower((string)$scheme), ['http', 'https'], true)) {
 return false;
 }
 $urlPath = (string)parse_url($url, PHP_URL_PATH);
 return (bool)preg_match('/\.(jpg|jpeg|png|webp|gif)$/i', $urlPath);
}
```
```

2. Gestione delle variabili di sessione

2.1 Avvio e configurazione della sessione

In `config.php` la sessione viene inizializzata con cookie più sicuri:

```
```php
if (session_status() !== PHP_SESSION_ACTIVE) {
 $secureCookie = !empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off';
 session_set_cookie_params([
 'lifetime' => 0,
 'path' => '/',
 'domain' => '',
 'secure' => $secureCookie,
 'httponly' => true,
 'samesite' => 'Lax',
]);
 session_start();
}
```
```

Questo riduce il rischio che il cookie di sessione venga letto o usato da script o altri siti.

2.2 Variabili di sessione nel login

Dopo l'autenticazione valida, il sistema salva informazioni importanti in `$_SESSION`:

```
```php
session_regenerate_id(true);
$_SESSION['id_utente'] = $id_utente;
$_SESSION['username'] = $username;
$_SESSION['ruolo'] = $ruolo_db ?: 'utente';
```
```

Il `session_regenerate_id(true)` impedisce la session fixation.

2.3 Controllo autorizzazioni

Più pagine controllano l'accesso controllando le variabili di sessione, ad esempio in `admin_prodotti.php`:

```
```php
if (!isset($_SESSION['id_utente'])) {
 header('Location: login.php?next=' . urlencode('admin_prodotti.php'));
 exit;
}
if (($_SESSION['ruolo'] ?? 'utente') !== 'admin') {
 // accesso negato per chi non è admin
}
```
```

E in `checkout.php`:

```
```php
if (!isset($_SESSION['id_utente'])) {
 header('Location: login.php?next=' . urlencode('checkout.php'));
 exit;
}
```
```

3. Password hash e confronto

3.1 Creazione hash sicuro

In ``registrazione.php``, la password viene salvata con ``password_hash()``:

```
```php
$password_salvata = password_hash($password, PASSWORD_DEFAULT);
```
```

Questo usa algoritmi moderni raccomandati da PHP e non memorizza mai la password in chiaro.

3.2 Verifica password durante il login

In ``login.php`` la verifica avviene con ``password_verify()``:

```
```php
if (is_string($password_db) && $password_db !== '' &&
 str_starts_with($password_db, '$')) {
 $ok = password_verify($password, $password_db);
 if ($ok && password_needs_rehash($password_db, PASSWORD_DEFAULT)) {
 $newHash = password_hash($password, PASSWORD_DEFAULT);
 }
} else {
 $ok = hash_equals((string)$password_db, $password);
 if ($ok) {
 $newHash = password_hash($password, PASSWORD_DEFAULT);
 }
}
```
```

3.3 Migrazione da password in chiaro

Il progetto supporta anche utenti legacy con password memorizzate in chiaro. Se la password corrisponde, viene rigenerato un hash sicuro e salvato:

```
```php
if ($ok) {
 $up = $conn->prepare("UPDATE utenti SET password = ? WHERE id_utente = ?");
 if ($up) {
 $up->bind_param("si", $newHash, $id_utente);
 $up->execute();
 $up->close();
 }
}
```
```

Questo aiuta a migliorare la sicurezza senza forzare subito tutti gli utenti a cambiare password.

4. Protezione CSRF

4.1 Generazione token CSRF

In `config.php` è definita la funzione per creare un token unico per il form:

```
```php
function csrf_token(): string
{
 if (!isset($_SESSION['csrf_token']) || !is_string($_SESSION['csrf_token']) ||
strlen($_SESSION['csrf_token']) < 32) {
 $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
 }
 return $_SESSION['csrf_token'];
}
```
```

4.2 Verifica token CSRF

Per verificare il token inviato dal form:

```
```php
function csrf_is_valid(?string $token): bool
{
 if (!is_string($token) || $token === '') {
 return false;
 }
 $sessionToken = $_SESSION['csrf_token'] ?? '';
 return is_string($sessionToken) && $sessionToken !== '' &&
hash_equals($sessionToken, $token);
}
```
```

4.3 Uso nei form sensibili

Un esempio in `checkout.php`:

```
```php
<input type="hidden" name="csrf_token" value="<?php echo
htmlspecialchars(csrf_token(), ENT_QUOTES, 'UTF-8'); ?>">
```
```

E la verifica al POST:

```
```php
if (!csrf_is_valid($_POST['csrf_token'] ?? null)) {
 $errore = 'Richiesta non valida. Ricarica la pagina e riprova.';
}
```
```

5. Protezione da SQL injection

Il progetto usa query preparate con `bind_param()` in molte parti importanti. Esempio da `login.php`:

```
```php
$stmt = $conn->prepare("SELECT id_utente, password, ruolo FROM utenti WHERE
username = ?");
$stmt->bind_param("s", $username);
```
```

Nella registrazione:

```
```php
$stmt = $conn->prepare("INSERT INTO utenti (username, email, password, ruolo)
VALUES (?, ?, ?, 'utente')");
$stmt->bind_param("sss", $username, $email, $password_salvata);
```
```

Nelle funzioni carrello di `config.php` sono usate preparazioni simili per tutte le query con parametri dinamici.

6. Altre parti importanti per interrogazione

6.1 Uso di `session_regenerate_id(true)`

Questa operazione cambia l'ID della sessione dopo il login per impedire che un attaccante riutilizzi il vecchio ID.

6.2 Controllo del ruolo utente

Le pagine admin controllano il ruolo tramite `$_SESSION['ruolo']`.

6.3 Validazione `next` per redirect sicuri

In `login.php` si controlla che il redirect `next` sia un file locale valido:

```
```php
if (!is_string($next) || !preg_match('/^[a-z0-9_\\-\\.]+\\.php$/i', $next)) {
 $next = 'index.php';
}
```
```

Questo evita redirect esterni malevoli.